

INFORME DE LABORATORIO 2

Expresiones Regulares con JFLAP

Asignatura	Teoría de la Computación INFO1148
Carrera	Ingeniería Civil en Informática
Docente	M. Lévano
Integrantes	<i>José Sáez y Catalina Vergara</i>
Fecha	11/10/26

1. Introducción

En este laboratorio trabajamos con expresiones regulares (ER) y con los autómatas que reconocen los lenguajes que ellas describen, usando JFLAP como herramienta de apoyo. El objetivo general era comprobar con ejemplos concretos que una ER, un autómata finito no determinista (AFN) y un autómata finito determinista (AFD) son tres formas distintas de representar la misma clase de lenguajes. Para eso recorrimos el camino completo: interpretar una ER dada, diseñar ER propias a partir de una descripción en palabras, pasar de ER a AFN con la construcción de Thompson y pasar de AFN a AFD con la construcción por subconjuntos.

El método fue el mismo en todas las preguntas. Antes de simular cada autómata anotamos a mano qué esperábamos para cada cadena de prueba, y recién después comparamos con lo que entregó JFLAP. Los autómatas se guardaron en formato .jff (JFLAP 7.1) y se adjuntan en la sección de Anexos.

2. Desarrollo

2.1. Pregunta 1 – Interpretación y validación de una expresión regular

a) Lenguaje representado

Sea $\Sigma = \{0, 1\}$ y la expresión regular:

$$(0+1)^*01$$

Esta expresión representa el conjunto de **todas las cadenas binarias que terminan en la subcadena 01**. En efecto, el factor $(0+1)^*$ genera un prefijo cualquiera sobre $\{0, 1\}$ —incluida la cadena vacía— y a continuación los símbolos literales 0 y 1 obligan a que las dos últimas posiciones de la cadena sean exactamente 0 y 1. La expresión no impone ninguna restricción sobre el contenido ni la longitud del prefijo, por lo que el lenguaje es:

$$L_1 = \{ w \in \{0, 1\}^* \mid w \text{ termina en } 01 \}.$$

b) Resultados obtenidos

La siguiente tabla registra la predicción (antes de ejecutar en JFLAP) y el resultado arrojado por el simulador.

Cadena	Predicción	Resultado JFLAP	¿Coinciden?
01	Aceptada	Aceptada	Sí
101	Aceptada	Aceptada	Sí
1101	Aceptada	Aceptada	Sí
0001	Aceptada	Aceptada	Sí
10	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
0100	Rechazada	Rechazada	Sí
10101	Aceptada	Aceptada	Sí

Cuadro 1: Resultados de la Pregunta 1.

c) Evidencia en JFLAP

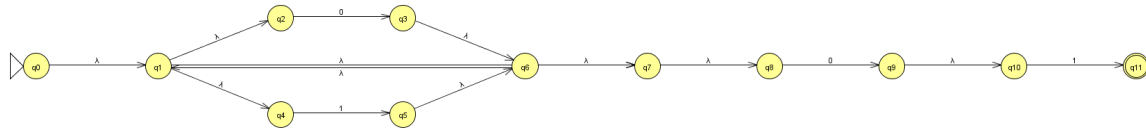


Figura 1: AFN obtenido en JFLAP para la expresión $(0+1)^*01$.

d) Análisis

El AFN generado por la construcción de Thompson a partir de la expresión $(0+1)^*01$ posee 12 estados, 14 transiciones (10 de ellas λ) y es no determinista, ya que utiliza transiciones vacías y presenta movimientos múltiples. Al simular las ocho cadenas, todas las predicciones coincidieron con el resultado de JFLAP.

La cadena 1101 es **aceptada** porque sus dos últimas posiciones son 01: el prefijo 11 es absorbido por $(0+1)^*$ y los símbolos finales 01 cierran la expresión, alcanzándose el estado final. En cambio, 0100 es **rechazada** porque termina en 00; aunque el primer 0 y el 1 intermedio podrían recorrerse, la expresión exige terminar en 01 y la cadena no lo cumple, por lo que no existe camino válido hacia el estado final. En general, el autómata acepta exactamente las cadenas que terminan en 01, confirmando la interpretación del lenguaje L_1 .

2.2. Pregunta 2 – Diseño de una expresión regular

a) Expresión regular propuesta

Lenguaje: $L_2 = \{ w \in \{a, b\}^* \mid w \text{ comienza con } a \text{ y termina con } b \}$.

$$a(a+b)^*b$$

b) Explicación

La expresión se diseña en tres partes con responsabilidades bien definidas:

- La **primera** a es un símbolo obligatorio y literal: obliga a que toda cadena aceptada *comience* con a .
- El factor central $(a+b)^*$ permite cualquier secuencia intermedia de a y b (incluso vacía), es decir, la *parte libre* de la cadena.
- La **última** b es un símbolo obligatorio y literal: obliga a que toda cadena aceptada *termine* con b .

Pero la combinación de ambas condiciones da más información: la primera a y la última b son literales que quedan fuera de la cerradura, por lo que toda cadena aceptada debe comenzar con a y terminar con b a la vez. La cadena de longitud mínima es ab (longitud 2).

c) Casos de prueba

Se probaron 15 cadenas: aceptadas, rechazadas, la de longitud mínima (ab) y casos límite (la cadena vacía ε , la cadena a , que comienza con a pero no termina en b , y la cadena b , que termina en b pero no comienza con a).

Cadena	Resultado esperado	Resultado JFLAP
ab	Aceptada	Aceptada
aab	Aceptada	Aceptada
abb	Aceptada	Aceptada
abab	Aceptada	Aceptada
aabb	Aceptada	Aceptada
aaab	Aceptada	Aceptada
abbb	Aceptada	Aceptada
a	Rechazada	Rechazada
b	Rechazada	Rechazada
ba	Rechazada	Rechazada
bbb	Rechazada	Rechazada
ε	Rechazada	Rechazada
abba	Rechazada	Rechazada
aba	Rechazada	Rechazada
bbbb	Rechazada	Rechazada

Cuadro 2: Casos de prueba de la Pregunta 2.

d) Evidencia en JFLAP

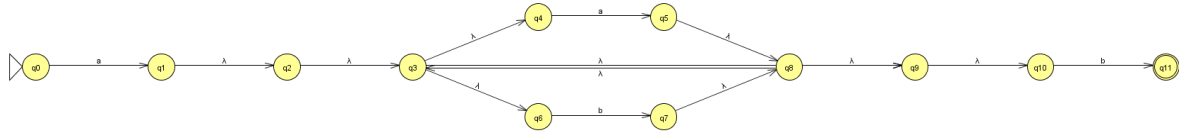


Figura 2: AFN generado en JFLAP para $a(a+b)^*b$.

e) Análisis

Todas las predicciones coincidieron con JFLAP. Las cadenas aceptadas comienzan con a y terminan con b (por ejemplo, $abab$ cumple ambas condiciones), mientras que las rechazadas fallan al menos una: a y aba comienzan con a pero no terminan en b ; b y ba no comienzan con a ; y ε falla por longitud. El AFN resultante tiene 12 estados y 14 transiciones (10 λ). Se concluye que $a(a+b)^*b$ describe de forma exacta el lenguaje L_2 .

2.3. Pregunta 3 – Conversión $ER \rightarrow AFN \rightarrow AFD$

a) Expresión regular propuesta

Lenguaje: $L_3 = \{ w \in \{0,1\}^* \mid w \text{ contiene la subcadena } 101 \}$.

$$(0+1)^*101(0+1)^*$$

La primera $(0+1)^*$ genera el prefijo previo a la subcadena 101; los símbolos 101 fuerzan la aparición de la subcadena buscada; y la segunda $(0+1)^*$ genera el sufijo posterior.

b) Características de los autómatas

Característica	AFN	AFD
Número de estados	22	9
Número de transiciones	27	18
Transiciones λ	Sí (20)	No
Determinista	No	Sí

Cuadro 3: Comparación estructural entre el AFN y el AFD de la Pregunta 3.

c) Pruebas realizadas

Cadena	Resultado AFN	Resultado AFD	¿Mismo resultado?
101	Aceptada	Aceptada	Sí
0101	Aceptada	Aceptada	Sí
1010	Aceptada	Aceptada	Sí
11011	Aceptada	Aceptada	Sí
000	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
1001	Rechazada	Rechazada	Sí
1101010	Aceptada	Aceptada	Sí

Cuadro 4: Pruebas de equivalencia entre AFN y AFD en la Pregunta 3.

d) Evidencias

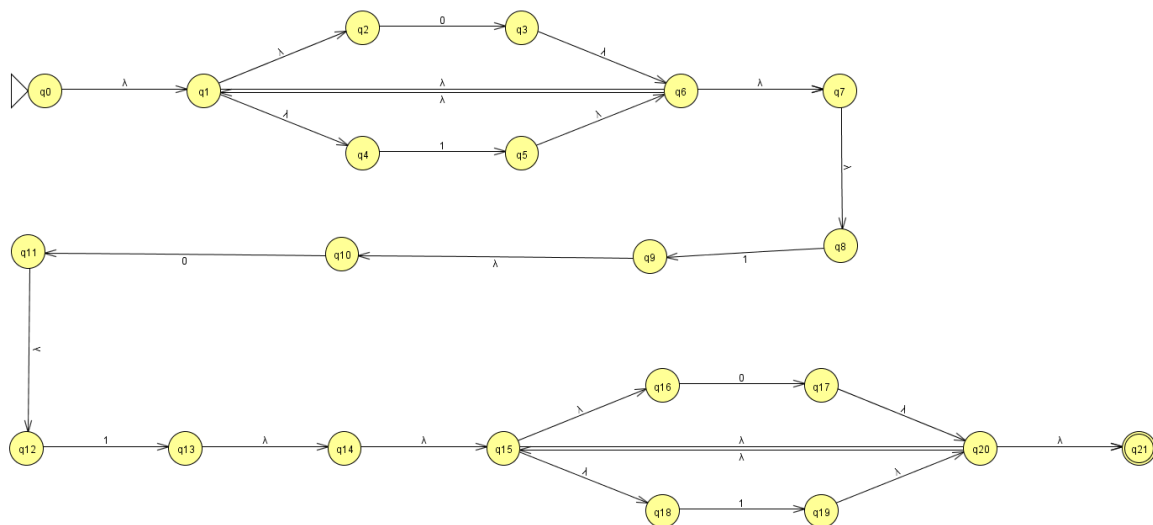


Figura 3: AFN generado a partir de la expresión $(0+1)^*101(0+1)^*$.

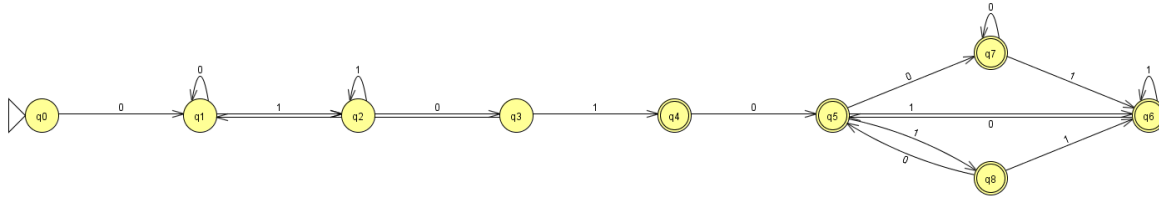


Figura 4: AFD obtenido por la construcción de subconjuntos a partir del AFN anterior.

e) Análisis

¿El AFN y el AFD reconocen lenguajes diferentes? **No.** Ambos reconocen el mismo lenguaje L_3 , es decir, todas las cadenas que contienen la subcadena 101. Esto se justifica de dos maneras. En primer lugar, *teóricamente*: la construcción por subconjuntos transforma un AFN en un AFD que acepta exactamente el mismo lenguaje; los estados del AFD son subconjuntos de estados del AFN que representan el progreso en la búsqueda de 101, por lo que no se añade ni se pierde ninguna cadena. En segundo lugar, *experimentalmente*: las ocho cadenas probadas arrojaron idéntico resultado en el AFN y en el AFD, incluidas las rechazadas 000, 111 y 1001. La diferencia entre ambos es únicamente representacional: el AFN usa 20 transiciones λ y puede tener varios movimientos para un mismo símbolo, mientras que el AFD no tiene transiciones λ y define una única transición por cada estado y símbolo. El AFD es, por tanto, una versión equivalente pero determinista del mismo reconocedor.

2.4. Pregunta 4 – Diseño y validación integrada

a) Expresión regular propuesta

Se desea reconocer las cadenas sobre $\Sigma = \{a, b\}$ que (1) contienen al menos una a y (2) terminan en bb .

$$(a+b)^*a(a+b)^*bb$$

b) Justificación

La expresión garantiza ambas condiciones mediante dos componentes claramente separados:

- **Al menos una a :** el bloque $(a+b)^*a(a+b)^*$ contiene un símbolo a *obligatorio* rodeado por secuencias arbitrarias de a y b . La a obligatoria puede ubicarse en cualquier posición de la cadena, lo que asegura la existencia de al menos una a .
- **Terminar en bb :** la parte final bb está fuera de toda cerradura, por lo que fuerza a que los dos últimos símbolos de la cadena sean b y b .

La estrategia consiste en separar la condición “contener una a ” (resuelta en el centro de la expresión) de la condición “terminar en bb ” (resuelta por el sufijo fijo), de modo que la concatenación las impone simultáneamente. La cadena aceptada más corta es abb (longitud 3).

c) Casos de prueba

Se probaron 14 cadenas: 6 aceptadas, 6 rechazadas y 2 casos límite definidos como ab (contiene a pero no termina en bb) y bba (contiene a y contiene la subcadena bb , pero no termina en bb).

Cadena	Resultado esperado	AFN	AFD
abb	Aceptada	Aceptada	Aceptada
aabb	Aceptada	Aceptada	Aceptada
babbb	Aceptada	Aceptada	Aceptada
ababbb	Aceptada	Aceptada	Aceptada
baaabb	Aceptada	Aceptada	Aceptada
abbb	Aceptada	Aceptada	Aceptada
bb	Rechazada	Rechazada	Rechazada
aaa	Rechazada	Rechazada	Rechazada
aba	Rechazada	Rechazada	Rechazada
baba	Rechazada	Rechazada	Rechazada
bbbb	Rechazada	Rechazada	Rechazada
ε	Rechazada	Rechazada	Rechazada
ab	Rechazada	Rechazada	Rechazada
bba	Rechazada	Rechazada	Rechazada

Cuadro 5: Casos de prueba de la Pregunta 4 (AFN y AFD coinciden en todos).

d) Evidencias

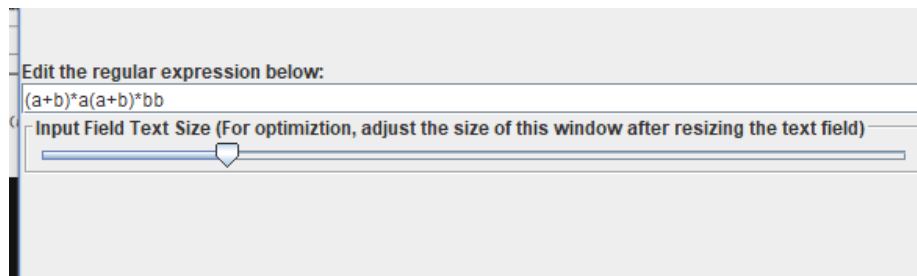


Figura 5: Expresión regular $(a+b)^*a(a+b)^*bb$ ingresada en JFLAP.

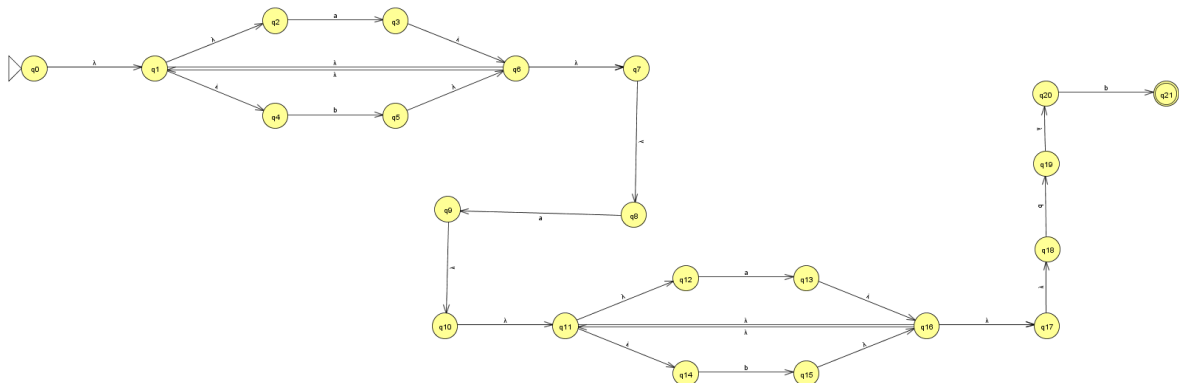


Figura 6: AFN generado para $(a+b)^*a(a+b)^*bb$.

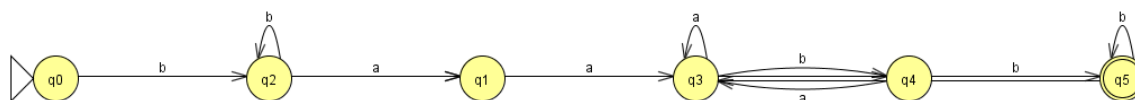


Figura 7: AFD obtenido por subconjuntos a partir del AFN anterior.

e) Reflexión

Las expresiones regulares, los AFN y los AFD son tres maneras distintas de describir lo mismo: los **lenguajes regulares**. Ninguna tiene más poder de reconocimiento que las otras. Según el teorema de Kleene, toda expresión regular se puede pasar a un AFN con la construcción de Thompson, y todo AFN se puede convertir en un AFD con la construcción por subconjuntos, y en ambos pasos el lenguaje se mantiene igual. Con las 14 cadenas de prueba lo comprobamos: el AFN y el AFD respondieron igual en todas. Lo que cambia es la comodidad de cada uno. El AFN, con sus transiciones λ y sus varios movimientos por símbolo, es más natural de obtener desde una expresión regular. El AFD, en cambio, tiene una sola transición por estado y símbolo, así que es más simple de simular e implementar.

3. Conclusiones

Este laboratorio nos ayudó a entender mejor la relación entre las expresiones regulares, los AFN y los AFD. Vimos que la construcción de Thompson convierte una ER en un AFN que reconoce el mismo lenguaje, pero a cambio aparecen las transiciones λ y el no determinismo. Después, con la construcción por subconjuntos, esos dos elementos desaparecen y se obtiene un AFD que acepta exactamente las mismas cadenas. En todas las preguntas, lo que habíamos predicho a mano coincidió con el resultado de JFLAP, lo que nos hizo sentir más seguros con el tema.

Otra cosa que notamos es que armar una expresión regular se parece a ir juntando piezas: cada condición del lenguaje (empezar con un símbolo, terminar con otro, contener una subcadena o combinar dos condiciones) corresponde a una parte específica de la expresión. Además, JFLAP fue una buena herramienta para comprobar nuestro trabajo, porque podíamos generar los autómatas, verlos y probar cadenas para ver si la teoría se cumplía al simular.

4. Archivos adjuntos

Junto con este informe se entregan los siguientes archivos de JFLAP (`.jff`), todos correspondientes a la versión presentada:

- P1_AFN.jff — AFN para $(0+1)^*01$.
- P2_AFN.jff — AFN para $a(a+b)^*b$.
- P3_AFN.jff — AFN para $(0+1)^*101(0+1)^*$.
- P3_AFD.jff — AFD obtenido del AFN de la Pregunta 3.
- P4_AFN.jff — AFN para $(a+b)^*a(a+b)^*bb$.
- P4_AFD.jff — AFD obtenido del AFN de la Pregunta 4.

5. Anexos

Enlace al repositorio (GitHub): https://github.com/saezjose/Lab02_Teoria_Computacion_

Uso de IA

Se utilizó IA como apoyo para revisar el informe y mejorar su redacción y detalles con los autómatas.